1. Node Markings in Code

Python Version

```python
def transform_numbers(n):
    # Node 1: Initialization
    i = 1
    total = 0
    checkpoints = 0

    while i <= n:  # Node 2: Main loop condition (Predicate 1)
        if i % 3 == 0:  # Node 3: Predicate 2
            total += i // 3  # Node 4
        elif i % 2 == 0:  # Node 5: Predicate 3
            total -= i // 2  # Node 6
        else:
            total += 2 * i  # Node 7

        if i % 4 == 0:  # Node 8: Predicate 4
            # Java equivalent: for(int j=0; j<2; j++)
            # Node 9: int j=0 (Initialization)
            # Node 10: j<2 (Condition / Predicate 5)
            # Node 12: j++ (Increment, executed after body)
            for j in range(2):
                # Node 11: Body
                print(f"Checkpoint at i={i}, j={j}")
                checkpoints += 1

        if i % 5 == 0:  # Node 13: Predicate 6
            continue  # Node 14

        i += 1  # Node 15

# Node 16: Exit
```

Java Version

```java
public static void transformNumbers(int n) {
    // Node 1: Initialization
    int i = 1;
    int total = 0;
    int checkpoints = 0;

    while (i <= n) { // Node 2: Main loop condition (Predicate 1)
        if (i % 3 == 0) { // Node 3: Predicate 2
            total += i / 3; // Node 4
```

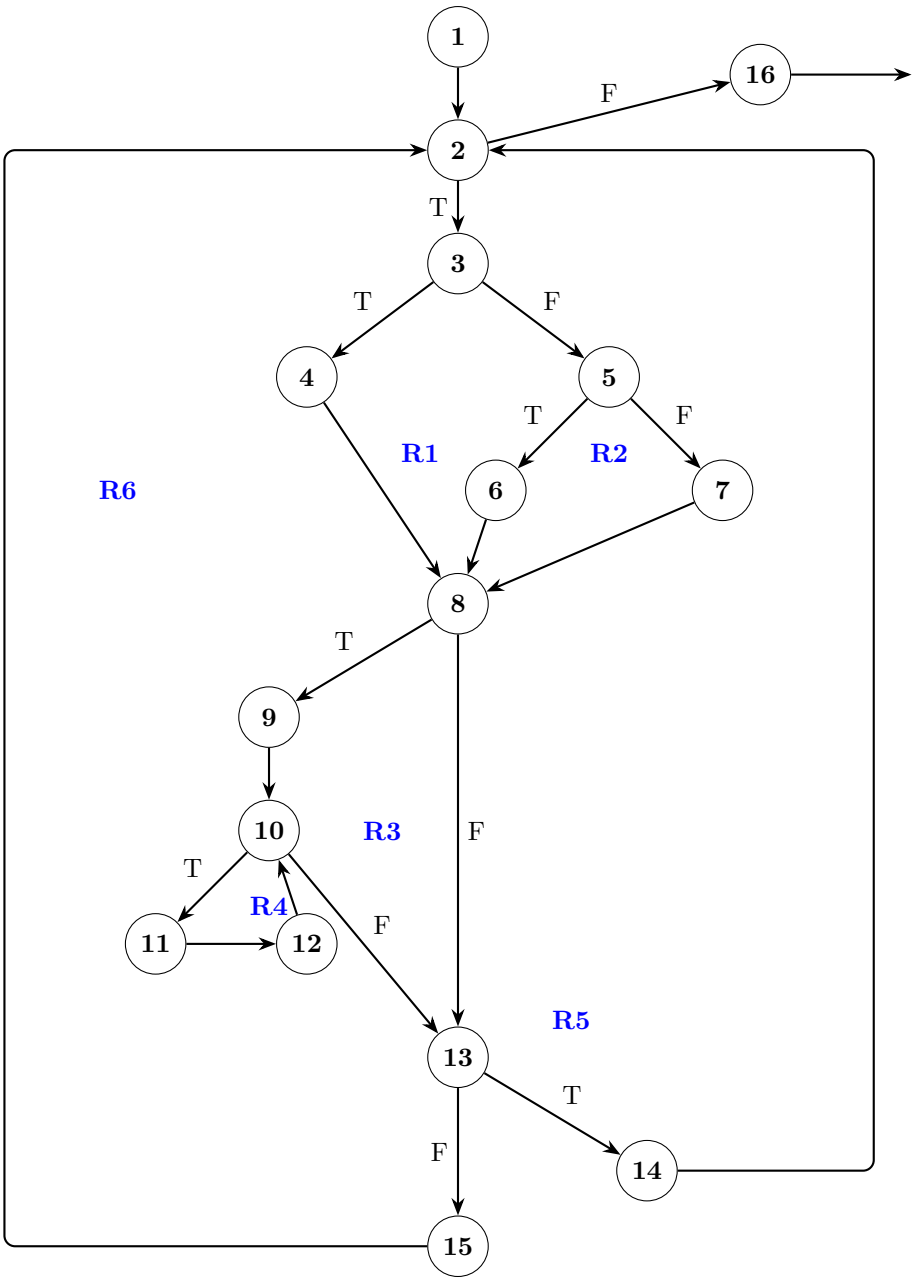
```
        } else if (i % 2 == 0) { // Node 5: Predicate 3
            total -= i / 2; // Node 6
        } else {
            total += 2 * i; // Node 7
        }

        if (i % 4 == 0) { // Node 8: Predicate 4
            // Node 9: int j = 0 (Initialization)
            // Node 10: j < 2 (Condition / Predicate 5)
            // Node 12: j++ (Increment, executed after body)
            for (int j = 0; j < 2; j++) {
                // Node 11: Body
                System.out.println("Checkpoint at i=" + i + ", j=" + j);
                checkpoints++;
            }
        }

        if (i % 5 == 0) { // Node 13: Predicate 6
            i++;
            continue; // Node 14
        }

        i++; // Node 15
    }
    // Node 16: Exit
}
```

2. Control Flow Graph (CFG)



3. Cyclomatic Complexity (M)

The Cyclomatic Complexity (M) is calculated using three standard formulas. All yield the same result:

1. **Using Regions (r):**
   $M = R + 1 = 6 + 1 = \mathbf{7}$
2. **Using Predicate Nodes (P):**
   The predicate (decision) nodes are 2, 3, 5, 8, 10, and 13. ($P = 6$)
   $M = P + 1 = 6 + 1 = \mathbf{7}$
3. **Using Edges and Nodes (E, N, P):**
   Where E = number of edges (21), N = number of nodes (16), and P = number of connected components (1).
   $M = E - N + 2P = 21 - 16 + 2(1) = 5 + 2 = \mathbf{7}$

4. Independent Paths

Below are the **7** independent paths (forming a basis set). Each subsequent path introduces at least one new edge.

1. **Path 1:** $1 \to 2 \to 16$
   (While loop condition is initially false; skips loop completely)
2. **Path 2:** $1 \to 2 \to 3 \to 4 \to 8 \to 13 \to 15 \to 2 \to 16$
   (Executes the '% 3 == 0' branch, bypasses the for-loop entirely, bypasses 'continue')
3. **Path 3:** $1 \to 2 \to 3 \to 5 \to 6 \to 8 \to 13 \to 15 \to 2 \to 16$
   (Executes the '% 2 == 0' branch)
4. **Path 4:** $1 \to 2 \to 3 \to 5 \to 7 \to 8 \to 13 \to 15 \to 2 \to 16$
   (Executes the fallback 'else' branch for the first condition block)
5. **Path 5:** $1 \to 2 \to 3 \to 4 \to 8 \to 9 \to 10 \to 13 \to 15 \to 2 \to 16$
   (Enters '% 4 == 0' block but evaluates for-loop condition $j < 2$ as false immediately)
6. **Path 6:** $1 \to 2 \to 3 \to 4 \to 8 \to 9 \to 10 \to 11 \to 12 \to 10 \to 13 \to 15 \to 2 \to 16$
   (Executes the inner for-loop body once before exiting the for-loop)
7. **Path 7:** $1 \to 2 \to 3 \to 4 \to 8 \to 13 \to 14 \to 2 \to 16$
   (Executes the '% 5 == 0' true branch, hitting the continue statement back to main loop condition)